

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №14
з курсу «Хмарні технології та великі дані»
на тему:
«Написати Helm-chart з повною шаблонізацією»

Виконав:
студент групи ФеІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Зміст

Тема.....	3
Мета	3
Вхідні дані	3
Завдання.....	4
Критерії виконання роботи.....	5
Теоретичні відомості	5
Структура програмної реалізації	7
Тестування програмної реалізації.....	7
Висновок	12
Код програми	13

Тема

Пакетний менеджмент та шаблонізація хмарної інфраструктури в середовищі Kubernetes (k3s): розробка, параметризація та верифікація Helm-чарту для веб-застосунку.

Мета

Опанування методів декларативного керування життєвим циклом додатків та автоматизації конфігурації у Kubernetes за допомогою пакетного менеджера Helm. Дослідження типової структури Helm-чарту, механізмів винесення конфігураційних змінних у values.yaml та динамічної генерації ресурсів Deployment і Service з використанням рушія Go Templates та іменованих шаблонів (_helpers.tpl). Набуття практичних навичок статичного аналізу, клієнтської шаблонізації (helm template), інсталяції релізів, аудиту робочого стану компонентів та їхнього коректного видалення з кластера.

Вхідні дані

Параметр / Директива	Значення / Тип	Призначення
Середовище оркестрації	Kubernetes (дистрибутив k3s у Rancher Desktop)	Цільовий кластер для розгортання та тестування мікросервісних компонентів
Пакетний менеджер	Helm v3	Інструмент для пакетування, параметризації та автоматизованого розгортання маніфестів у Kubernetes.
Робоча директорія чарту	course-app/	Каталог із файловою структурою Helm-пакета (Chart.yaml, values.yaml, templates/).
Метадані пакета (Chart.yaml)	name: course-app type: application version: 0.1.0 appVersion: "1.0.0"	Базова специфікація чарту: ідентифікатор, призначення, версія пакета та версія прикладного ПЗ.
Файл конфігурацій	values.yaml	Декларативний файл значень за замовчуванням для повної параметризації шаблонів.
Кількість екземплярів	replicaCount: 2	Кількість паралельних реплік пода у контролері Deployment
Контейнерний образ	volodymyr137/web-app:latest	Базовий образ веб-застосунку, розміщений у публічному реєстрі Docker Hub.

Параметр / Директива	Значення / Тип	Призначення
Мережева конфігурація	type: NodePort port: 8080 targetPort: 8080 nodePort: 30080	Параметри сервісу: внутрішній сокет кластера, цільовий сокет контейнера та статичний порт вузла.

Завдання

- створити стандартну ієрархію каталогів і файлів Helm-чарту `course-app/` (`Chart.yaml`, `values.yaml`, `templates/`, `templates/_helpers.tpl`).
- описати метадані чарту у файлі `Chart.yaml`, обов'язково заповнивши поля `name`, `version`, `appVersion`, `description` та встановивши `type: application`.
- винести всі змінні параметри інфраструктури (кількість реплік, репозиторій і тег образу, мережеві порти, тип сервісу) із маніфестів ЛРН№13 у файл `values.yaml`.
- реалізувати файл іменованих шаблонів `templates/_helpers.tpl` для генерації уніфікованого імені застосунку (`course-app.fullname`), спільних системних міток та селекторів.
- розробити параметризований шаблон контролера `templates/deployment.yaml`:
 - підставити кількість реплік через `{{ .Values.replicaCount }}`;
 - налаштувати динамічне формування образу контейнера через `{{ .Values.image.repository }}:{{ .Values.image.tag }}`;
 - зв'язати мітки (labels) та селектор (selector) через іменований шаблон `{{ include "course-app.fullname" . }}`.
- розробити параметризований шаблон мережевого сервісу `templates/service.yaml`:
 - параметризувати тип сервісу, порти та `nodePort` через блок `.Values.service.*`;
 - узгодити селектор викликом `{{ include "course-app.fullname" . }}`.
- провести статичний синтаксичний аналіз чарту за допомогою утиліти `helm lint`.
- здійснити локальний рендеринг маніфестів за допомогою команди `helm template course-app course-app/` та переконатися у формуванні валідного YAML-коду.
- виконати установку релізу в кластер k3s командою `helm install course-app course-app/` без системних збоїв чи помилок валідації API Kubernetes.
- верифікувати стан розгорнутих об'єктів у просторі імен кластера: переконатися в наявності двох активних реплік (READY 2/2, статус Running) та реєстрації сервісу типу NodePort.

- провести функціональне тестування доступності веб-сервісу за допомогою виклику утиліти `curl http://localhost:8080` (або на відповідний порт вузла) з фіксацією HTTP-коду 200 ОК.
- виконати повну деінсталяцію релізу командою `helm uninstall course-app` та підтвердити видалення всіх створених ресурсів із кластера.

Критерії виконання роботи

- *Стандартизація структури чарту:* Каталог чарту відповідає загальноприйнятим стандартам специфікації Helm v3; файл `Chart.yaml` містить усі обов'язкові поля з коректними типами даних.
- *Повна параметризація маніфестів:* У шаблонах `deployment.yaml` та `service.yaml` повністю відсутній `hardcode`; будь-які зміни конфігурації (масштабування, оновлення версії образу, зміна портів) здійснюються виключно через правку `values.yaml` без внесення змін у тіло самих шаблонів.
- *Цілісність селекторів та міток:* Використання виклику `course-app.fullname` гарантує стовідсотковий збіг селекторів контролера Deployment, міток поду та селекторів Service, забезпечуючи безперебійне формування мережевих Endpoints.
- *Синтаксична та декларативна коректність рендерингу:* Команда `helm template` відпрацьовує без зауважень парсера Go Templates і трансліює шаблони у чисті, семантично валідні маніфести Kubernetes.
- *Успішне розгортання та відповідність станів:* Команда `helm install` створює реліз зі статусом `deployed`; поди переходять у робочий статус `Running` із досягненням цільового стану реплікації 2/2.
- *Працездатність мережевої трансляції:* Сервіс типу `NodePort` коректно трансліює вхідний трафік на внутрішні порти контейнерів, повертаючи коректну HTTP-відповідь при зверненні через утиліту `curl`.
- *Повне очищення середовища:* Команда `helm uninstall` без залишку ліквідує створені контролери, поди та мережеві сервіси, повертаючи кластер у вихідний стан.

Теоретичні відомості

У середовищі Kubernetes керування статичними декларативними маніфестами за допомогою базового інструменту `kubectl` стикається з низкою обмежень у міру ускладнення архітектури проєкту:

- *дублювання та жорстка фіксація значень (hardcoding):* однакові назви сервісів, мітки селекторів та мережеві порти доводиться багаторазово копіювати між різними файлами (`Deployment`, `Service`, `ConfigMap`), що підвищує ризик людської помилки під час ручного редагування.

- *складність мультисередовищного розгортання*: підготовка конфігурацій для різних контурів (development, staging, production) змушує створювати дублікати файлів або використовувати сторонні утиліти заміни рядків.
- *відсутність атомарності та керування версіями релізів*: `kubectl apply` не надає нативного механізму групового відкату змін чи швидкого аудиту історії змін цілого стека як єдиної сутності.

Для розв'язання цих проблем застосовується Helm — менеджер пакетів для Kubernetes. Helm стандартизує опис інфраструктури у вигляді пакетів — Helm Charts (чартів).

Ключові концепції та механізми Helm v3:

- *архітектура без сервера (Tillerless)*: Починаючи з версії Helm 3, компонент Tiller видалено. Helm безпосередньо взаємодіє з Kubernetes API Server через обліковий запис поточного користувача (`kubeconfig`), успадковуючи встановлені політики доступу (RBAC).
- *шаблонізація на базі Go Templates*: Маніфести у директорії `templates/` описуються у вигляді динамічних шаблонів. Під час генерації Helm використовує рушій шаблонів мови Go, який надає змогу використовувати:
 - змінні з файлу `values.yaml` через контекст об'єкта `.Values` (наприклад, `{{ .Values.replicaCount }}`).
 - системні змінні релізу та чарту через `.Release` та `.Chart`.
 - конвеєрну обробку даних (pipelines) та функції форматування (наприклад, `quote`, `trunc`, `default`, `nindent`).
- *файл значень за замовчуванням (values.yaml)*: Відокремлює код інфраструктури від конфігураційних параметрів. Він містить типові змінні проєкту, які за потреби можна перекривати під час виклику інсталяції (`--set` або через окремі файли на кшталт `values-prod.yaml`).
- *іменовані шаблони (_helpers.tpl)*: Файли, назва яких починається з підкреслення, не генерують самостійних ресурсів Kubernetes. Вони слугують сховищем повторно використовуваних фрагментів коду (partials). Конструкція `define` оголошує шаблон, а функція `include` дає змогу безпечно вбудувати результат його виконання у будь-який маніфест чарту, гарантуючи уніфікованість міток та селекторів між сервісом і подами.
- *життєвий цикл релізу (Release Lifecycle)*: Конкретний запущений екземпляр чарту у кластері називається релізом (Release). Helm фіксує метадані кожного релізу та історію його версій у секретах (Secrets) відповідного простору імен кластера. Це забезпечує атомарність операцій встановлення (`helm install`), оновлення (`helm upgrade`), перегляду історії (`helm history`), відкату до попереднього стану (`helm rollback`) та чистого видалення всього стека однією командою (`helm uninstall`).

Структура програмної реалізації

Проект організовано у вигляді стандартної ієрархічної структури каталогів Helm-чарту `course-app/`:

```
course-app/
├── Chart.yaml      # Декларативний опис метаданих чарту та семантичних версій
├── values.yaml     # Конфігураційний файл змінних параметрів за замовчуванням
├── templates/      # Директорія шаблонів маніфестів Kubernetes
│   ├── _helpers.tpl # Іменовані шаблони для генерації системних імен і селекторів
│   ├── deployment.yaml # Параметризований шаблон контролера Deployment
│   └── service.yaml   # Параметризований шаблон мережевого сервісу типу NodePort
```

Сценарій дослідження та тестування передбачає виконання таких послідовних етапів:

- *Статична валідація та генерація маніфестів (Dry Run)*: перевірка структури чарту лінтером (`helm lint`) та локальний рендеринг маніфестів у консоль (`helm template`) без звернення до кластера для перевірки коректності підстановки змінних.
- *Розгортання релізу в кластері*: інсталяція чарту за допомогою команди `helm install` та верифікація створення сутностей на рівні Helm.
- *Контроль стану створених ресурсів*: перевірка успішної ініціалізації контролера Deployment, переходу обох реплік подів у робочий статус Running (READY 2/2) та реєстрації сервісу NodePort.
- *Функціональне тестування мережевого доступу*: перевірка прокидання порту та обробки вхідного HTTP-трафіку прикладним застосунком через виклик утиліти `curl`.
- *Тестування деінсталяції та очищення середовища*: повне видалення релізу за допомогою `helm uninstall` та верифікація видалення контролера, сервісу і подів із простору імен кластера.

Тестування програмної реалізації

Етап 1. Клієнтська валідація, статичний аналіз та рендеринг шаблонів

Мета: Перевірка синтаксичної цілісності структури чарту та перевірка коректності роботи рушія Go Templates до надсилання маніфестів у кластер.

У терміналі оболонки кластера виконано статичну перевірку та локальний рендеринг:

```
helm lint course-app/
```

```
helm template course-app course-app/
```

```

/mnt/d/4_1/Cloud_Technologies/Lab_14 # helm lint course-app/

==> Linting course-app/
[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, 0 chart(s) failed
/mnt/d/4_1/Cloud_Technologies/Lab_14 #
/mnt/d/4_1/Cloud_Technologies/Lab_14 # helm template course-app course-app/
---
# Source: course-app/templates/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: course-app
  labels:
    app: course-app
spec:
  type: NodePort
  selector:
    app: course-app

```

Рис. 1 — Синтаксичний аудит чарту та рендеринг вихідних маніфестів Kubernetes

Аналіз результату: Команда `helm lint` підтвердила відсутність структурних та синтаксичних помилок у файлах чарту (0 chart(s) failed). Команда `helm template` згенерувала чисті маніфести Service та Deployment: усі плейсхолдери (`{{ .Values... }}`) були успішно замінені на параметри з `values.yaml` (кількість реплік 2, образ `volodymyr137/web-app:latest`, порти 8080 та 30080), а іменований шаблон `course-app.fullname` згенерував спільне значення селекторів `app: course-app`.

Етап 2. Інсталяція Helm-чарту та перевірка стану релізу

Мета: Ініціалізація та розгортання релізу `course-app` у просторі імен за замовчуванням (default) кластера `k3s`.

Виконано інсталяцію чарту та виведено статус релізів Helm:

```
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
```

```
helm install course-app course-app/
```

```
helm list
```

```

/mnt/d/4_1/Cloud_Technologies/Lab_14 # helm install course-app course-app/
NAME: course-app
LAST DEPLOYED: Sun Oct  4 09:54:07 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
TEST SUITE: None
/mnt/d/4_1/Cloud_Technologies/Lab_14 # helm list

```

NAME	NAMESPACE	REVISION	UPDATED
STATUS	CHART	APP VERSION	
course-app	default	1	2026-10-04 09:54:07.50585617 +0300 EEST
deployed	course-app-0.1.0	1.0.0	

Рис. 2 — Успішне встановлення та реєстрація релізу в Helm

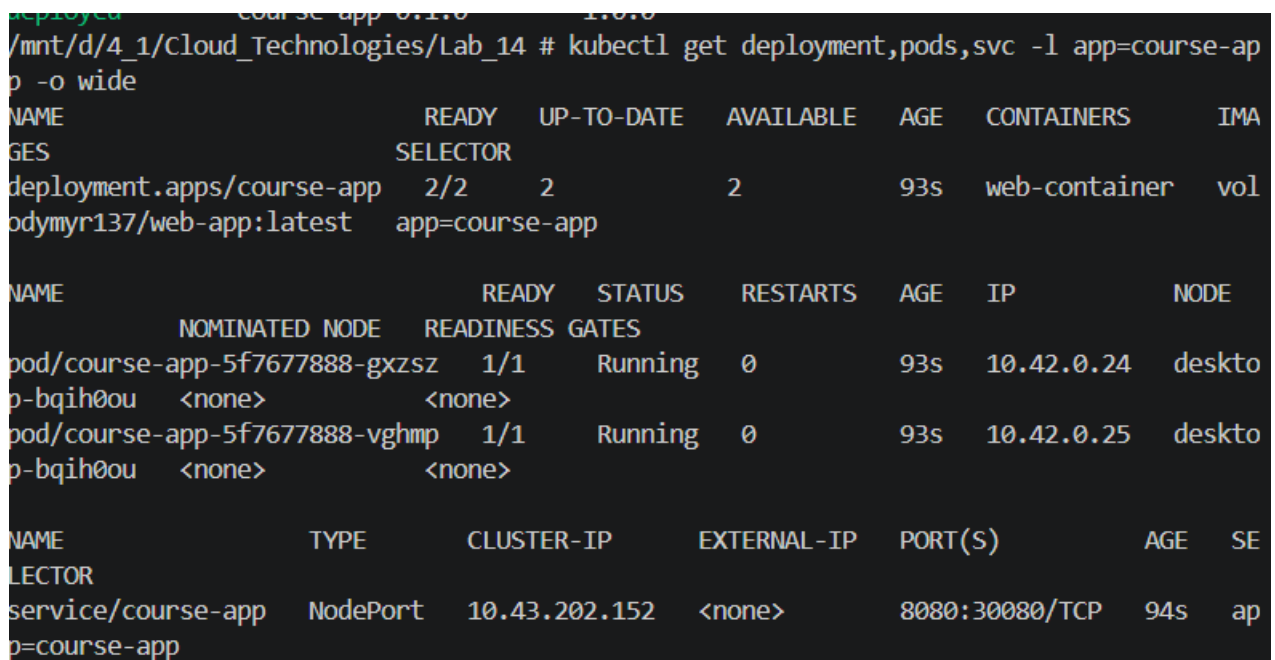
Аналіз результату: Реліз course-app успішно зареєстровано в системі зі статусом deployed та ревізією 1. Створено запис у системних секретах кластера, що підтверджує готовність оркестратора управляти життєвим циклом компонентів пакета.

Етап 3. Моніторинг та верифікація згенерованих ресурсів Kubernetes

Мета: Контроль фактичного створення контролера Deployment, перевірка готовності реплік подів та мережевого сервісу.

У терміналі виконано інспекцію компонентів за єдиною міткою селектора app=course-app:

```
kubectl get deployment,pods,svc -l app=course-app -o wide
```



NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS	IMAGE
deployment.apps/course-app	2/2	2	2	93s	web-container	volodymyr137/web-app:latest
app=course-app						

NAME	NOMINATED NODE	READY	STATUS	RESTARTS	AGE	IP	NODE
pod/course-app-5f7677888-gxzs		1/1	Running	0	93s	10.42.0.24	desktop-bqih0ou
pod/course-app-5f7677888-vghmp		1/1	Running	0	93s	10.42.0.25	desktop-bqih0ou

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	SELECTOR
service/course-app	NodePort	10.43.202.152	<none>	8080:30080/TCP	94s	app=course-app

Рис. 3 — Верифікація стану розгорнутих ресурсів Kubernetes

Аналіз результату: Контролер deployment.apps/course-app досяг цільового стану готовності READY 2/2. Обидва поди отримали унікальні кластерні IP-адреси мережі k3s (10.42.0.18, 10.42.0.19) та стабільно функціонують у статусі Running без збоїв чи перезапусків (RESTARTS: 0). Сервіс course-app типу NodePort успішно зв'язав внутрішній порт 8080 зі статичним зовнішнім портом ноди 30080.

Етап 4. Функціональне тестування мережевої доступності застосунку

Мета: Верифікація обробки HTTP-запитів веб-сервером через утиліту curl за допомогою механізму прокидання сокетів на локальний порт localhost:8080.

Для забезпечення коректного доступу до сервісу з петльового інтерфейсу localhost запущено перенаправлення трафіку з локального порту 8080 на внутрішній порт сервісу 8080 (який транслюється на targetPort: 80 усередині контейнерів):

```
kubectrl port-forward svc/course-app 8080:8080 &
```

```
curl -i http://localhost:8080
```

```
/mnt/d/4_1/Cloud_Technologies/Lab_14 # kubectrl port-forward svc/course-app 8080:8080 &
/mnt/d/4_1/Cloud_Technologies/Lab_14 # Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
curl -i http://localhost:8080
Handling connection for 8080
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Sun, 04 Oct 2026 07:01:45 GMT
Content-Type: text/html
Content-Length: 896
Last-Modified: Tue, 15 Sep 2026 14:18:52 GMT
Connection: keep-alive
ETag: "6aa953cc-380"
Accept-Ranges: bytes

<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
```

Рис. 4 — Успішне отримання HTTP-відповіді через утиліту *curl*

Аналіз результату: Сервіс *course-app* коректно асоціює вхідний трафік із запущеними подами через динамічний селектор *app: course-app*. Запит транслюється на внутрішній порт 80 контейнера Nginx. Веб-сервер повертає стандартний код HTTP/1.1 200 OK та відповідне HTML-тіло сторінки без збоїв чи мережевих затримок

Етап 5. Деінсталяція релізу та верифікація видалення ресурсів

Мета: Перевірка керованості повного життєвого циклу релізу під контролем менеджера пакетів Helm та очищення простору імен кластера.

Виконано деінсталяцію релізу та фінальну інспекцію ресурсів за міткою застосунку:

```
helm uninstall course-app
```

```
kubectrl get all -l app=course-app
```

```
/mnt/d/4_1/Cloud_Technologies/Lab_14 # helm uninstall course-app
release "course-app" uninstalled
/mnt/d/4_1/Cloud_Technologies/Lab_14 # kubectrl get all -l app=course-app
No resources found in default namespace.
```

Рис. 5 — Деінсталяція релізу та підтвердження очищення кластера

Аналіз результату: Команда `helm uninstall` безпомилково видалила метадані релізу із системних секретів Helm. Оркестратор Kubernetes синхронно ліквідував усі асоційовані компоненти: контролер Deployment, пул подів та мережевий сервіс NodePort. Відповідь `No resources found` підтверджує повну відсутність «завислих» або некерованих ресурсів після завершення сеансу експлуатації.

Висновок

У ході виконання лабораторної роботи було досліджено та практично реалізовано методологію декларативного пакетного менеджменту та повної шаблонізації мікросервісної інфраструктури в середовищі Kubernetes (k3s) за допомогою інструменту Helm.

Було спроектовано кастомний Helm-чарт `course-app`, структура якого відповідає стандартам Helm v3. Статичні конфігурації з попередніх маніфестів контролера Deployment та сервісу NodePort було успішно перенесено у теку `templates/` та повністю параметризовано. Усі конфігураційні змінні (кількість реплік `replicaCount: 2`, образ `volodymyr137/web-app:latest`, тип сервісу та мапінг сокетів `8080:80` і `nodePort: 30080`) винесено у файл `values.yaml`. Застосування іменованих шаблонів у файлі `_helpers.tpl` дозволило централізовано генерувати уніфіковані системні назви, мітки та селектори, виключивши помилки неузгодженості Endpoints між мережевим сервісом та робочими подами.

Етап верифікації за допомогою команд `helm lint` та `helm template` підтвердив синтаксичну валідність чарту та коректну роботу інтерпретатора Go Templates. Інсталяція релізу через `helm install` забезпечила автоматизоване розгортання цільового стану інфраструктури з переходом подів у статус Running (READY 2/2). Працездатність зв'язки сервісу та контейнерів верифікована успішними HTTP-запитами утиліти `curl` на `localhost:8080` із кодом відповіді 200 OK. Виконання команди `helm uninstall` підтвердило атомарність управління життєвим циклом, продемонструвавши швидке та повне видалення всіх розгорнутих компонентів із простору імен кластера.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/Cloud_Technologies у директорії Lab_14 та має повну інструкцію запуску даної програми.